

Ejercicios Node y Javascript

Metodología para publicar los cambios en el repositorio:
no esperes a terminar todos los ejercicios para realizar un solo commit con todos los ejercicios resueltos.

- a) Realizar una Rama llamada “develop” que partir de la rama Main o Master.
- b) Partir de develop y generar una rama para ejercicio 1, cuando termines de realizar el ejercicio 1 y commitees los cambios, mergear esa rama a develop.
- c) Partir de develop nuevamente (después del merge vas a tener todos los cambios realizados del ejercicio1), repetir el paso “b” para cada uno de los ejercicios.

Ejercicio 01 – Crear un Paquete personalizado

- 1) Inicializar un paquete nuevo.
- 2) Completar los datos que solicita para inicializar el paquete.
- 3) Crear un archivo que se llame miIndex.js, establecer el archivo miIndex.js como archivo de entrada del paquete.

Ejercicio 02 – Módulos externos

- 1) Instalar el paquete “Colors” para poder imprimir en consola mensajes con colores y más divertidos!
<https://www.npmjs.com/package/colors> (por si no recuerdan, en la documentación del link les dice como instalarlo y algunos ejemplos de cómo implementarlo)
- 2) Instalar el paquete express.
- 3) Momento! Aún no necesitamos el paquete express, desinstalarlo del proyecto.

Ejercicio 03 – Promesas

Implementar el módulo “Colors” del Ejercicio de módulos externos para imprimir los mensajes de consola con colores:

- rojo en caso de mensaje de error.
- verde en caso de mensaje de éxito.
- amarillo en caso de mensajes de información.

- 1) Crear un archivo llamado `cuentaBancaria.js`, dentro de este archivo realizar los siguientes métodos.
 - a) Se desea generar un método para realizar una transferencia y un método para validar el saldo disponible en la cuenta.
 - b) El método de realizar transferencia no se debe ejecutar hasta no validar el saldo y que haya saldo suficiente.
 - c) Simular retardos en los métodos de validar saldo y realizar transferencia de 5 segundos y 10 segundos respectivamente.
 - d) En cualquiera de los casos se debe ejecutar un log en consola que informe que se finalizó la operación. (había un método de la promesa que le paso un callback que se ejecuta en cualquiera de los dos casos)
 - e) Ejecutar las promesas y definir como se debe comportar en cada caso (resuelto/rechazado) pasándole las callbacks necesarias.

- 2) Crear un archivo llamado `instituto.js`, dentro de este archivo crear un objeto que se llame `alumno` con las siguientes propiedades: `Nombre`, `Edad`, `inscriptoAMaterias` (un array de strings), `debeCorrelativa` (un booleano)

realizar los siguientes métodos.

- a) Se desea generar un método para realizar una inscripción a una materia (recibe como parámetro de entrada el objeto `alumno`, y el nombre de la materia a inscribirse) y un método para validar si tiene las correlativas aprobadas (recibe como parámetro de entrada el objeto `alumno`)
- b) El método para realizar una inscripción no se debe ejecutar hasta no validar si tiene las correlativas aprobadas. En el método `validarCorrelativa`, utilizar la propiedad “`debeCorrelativa`” para validarlo, en el método para inscribir, en caso de que el alumno no deba la correlativa, agregar la materia al array de materias “`inscriptoAMaterias`” del objeto.
- c) Simular retardos en los métodos de validar correlativas y realizar inscripción de 2 segundos y 5 segundos respectivamente.
- d) En cualquiera de los dos casos se debe ejecutar un log en consola que informe que se finalizó la operación.
- e) Ejecutar las promesas y definir como se debe comportar en cada caso (resuelto/rechazado)

- 3) Crear un archivo llamado `mercadoRestringido.js`, dentro de este archivo generar cinco objetos de tipo `producto`, deben tener las propiedades: `id`, `nombreProducto`, `precio`, `stock`. Guardar los cinco objetos en un array llamado `productos`.

- a) Se desea generar un método para realizar venta, validar stock y generar etiqueta de envío. El método `validarStock` recibe como parámetros de entrada un nombre de producto a buscar y el array

- productos, debe buscar el producto, en el array de productos, en caso de que lo encuentre, debe validar el stock disponible. El método realizarVenta, debe recibir como parámetro de entrada un objeto producto, y en caso de que se ejecute la venta, descontar la cantidad vendida del producto. El método imprimirEtiqueta, recibe como parámetro de entrada el nombre del producto.
- b) El método para realizar venta no se debe ejecutar hasta no validar si hay stock suficiente.
 - c) El método generar etiqueta solo se debe ejecutar si se realizó la venta correctamente.
 - d) Simular retardos en los métodos de validar correlativas y realizar inscripción de 2 segundos y 1 segundos y 4 segundos respectivamente.
 - e) En cualquiera de los casos se debe ejecutar un log en consola que informe que se finalizó la operación.
 - f) Ejecutar las promesas y definir como se debe comportar en cada caso (resuelto/rechazado)

Ejercicio 04 – Módulos internos.

- a) Mover las implementaciones de las ejecuciones de los ejercicios 1.e, 2.e y 3.f, cada ejecución, dentro de una función para poder invocarlas llamando directamente a esas funciones.
- b) Exportar esas funciones para poder invocarlas en otros módulos.
- c) Importar las funciones del punto 4.a y 4.b en el archivo miIndex.js
- d) Invocar las funciones importadas con datos de ejemplo.

ENTREGA: Realizar la entrega en el aula virtual, compartiendo el link del repositorio donde se subió la práctica.

NOTA AL PIE:

Recordar las buenas prácticas de programación que fuimos viendo en clase!